

consensus

Raft Consensus Subsystem (pkg/raft + helix-raftd)

Field	Value
Status	active
Scope	pkg/raft/*, cmd/helix-raftd/*, specs/ RaftSnapshot.tla
Last reviewed against code	2026-06-13

This document describes the embedded Raft consensus subsystem of Helix Cluster OS as it exists in the codebase today. Every claim below maps to a named symbol, test, or spec. It deliberately describes **only what is implemented** — there is no multi-region, auto-sharding, or dynamic-membership layer in this subsystem, and none is claimed here.

The subsystem is built on the battle-tested github.com/hashicorp/raft library (plus github.com/hashicorp/raft-boltdb/v2 for durable storage). `pkg/raft` wraps that library behind a small, stable contract and grows it along a **transport/storage ladder**, from an in-process test cluster up to a real multi-process daemon. `cmd/helix-raftd` is the capstone process that runs one production-grade node.

1. The replicated state machine and the Node/Cluster contract

1.1 KVFSM — the replicated key/value state machine (fsm.go)

KVFSM implements `hashicorp/raft.FSM`. It is a `map[string]string` guarded by a `sync.RWMutex`. Replicated mutations are Command values:

```
type Command struct {  
    Op      CommandType // "set" | "delete"  
    Key     string  
    Value   string  
}
```

A Command is JSON-encoded (`Command.Encode`), appended to the Raft log, replicated to a majority, and only then applied to **every** replica's FSM in identical order by KVFSM.Apply. Read/inspection surface:

- `Get(key) (string, bool)` — concurrent-safe point read of this replica's state.
- `Len() int` — number of keys.
- `AppliedCount() uint64` — count of log entries applied to **this** FSM; a sink-side replication signal that, on a follower, reflects entries replicated and committed regardless of which node proposed them.

For log compaction the FSM implements Snapshot/Restore: Snapshot returns an immutable JSON clone (`kvSnapshot.Persist` writes it to the snapshot sink), Restore replaces state from a snapshot stream. This is the FSM half of the snapshot/compaction path that `specs/RaftSnapshot.tla` verifies (see §4.2).

1.2 Node — the per-node coordination wrapper (`node.go`)

Node wraps one `hashicorp/raft.Raft` instance together with its KVFSM, exposing a surface that intentionally resembles `pkg/Leader`'s election so the two backends are interchangeable from a caller's point of view:

Method	Behaviour
<code>Apply(cmd, timeout) error</code>	Propose a Command. Returns <code>ErrNotLeader</code> if not leader; otherwise blocks until the entry is replicated to a majority and applied, surfacing any FSM rejection.
<code>IsLeader() bool</code>	True iff <code>raft.State() == Leader</code> .
<code>State() hraft.RaftState</code>	Current Raft state (Leader/Follower/Candidate/Shutdown).
<code>Leader() string</code>	ServerID of the known leader, or "".
<code>LeaderAddr() string</code>	Network address of the known leader (a <code>TCP host:port</code> for TCP-backed nodes) — what a follower's admin endpoint reports so a client can redirect a write.
<code>LastIndex() uint64</code>	Last index in stable storage; a catch-up signal a restarted node reports as it replays its on-disk log.
<code>Shutdown() error</code>	Stops Raft, then closes the TCP transport (if any), then closes the on-disk store (if any) — in that order, so files flush and the data dir can be reopened.

A Node holds at most one transport: `trans` (in-mem) **xor** `netTransport` (TCP). Storage fields (`dataDir`, `closeStore`) are orthogonal — a persistent node sets them in addition to whichever transport it uses.

1.3 Cluster — the multi-node helper (cluster.go)

Cluster is a set of `*Node` plus polling helpers used by tests and builders:

- `Leader()` — the node that both reports Leader state **and** advertises itself as leader (guards against transient split views).
 - `WaitForLeader(timeout) / WaitForNewLeader(excludeID, timeout)` — poll until a (new) leader is observed; assert on observed state rather than sleeping a fixed duration.
 - `Survivors(excludeID)` — all nodes except one (used after killing a leader).
 - `Shutdown()` — stop every node, skipping already-stopped ones.
-

2. The transport/storage ladder

Each rung below adds exactly one production property over the previous one, over the **same** KVFSM/Node/Cluster contract. Shared wiring is factored out so the rungs cannot drift.

Rung	Constructor	Transport	Storage	D
1. In-mem	<code>NewInmemCluster(base, ids...)</code>	InmemTransport (in-process channel mux)	in-mem log/ stable/snapshot	ba (c
2. Real TCP	<code>NewNetworkCluster(base, ids...)</code>	real loopback NetworkTransport	in-mem	H (t
3. On-disk	<code>NewPersistentNode(id, dataDir, base)</code>	in-mem	BoltDB + FileSnapshotStore	H (s
4. Combined	<code>NewPersistentNetworkNode(...)</code> / <code>NewPersistentNetworkCluster(ids, dataDirs)</code>	real TCP	BoltDB + snapshots	H 15 (p
5. Fixed-address (multi-process)	<code>NewPersistentNetworkNodeAt(id, dataDir, bindAddr, base, servers)</code>	real TCP at a fixed host:port	BoltDB + snapshots	H (p da

2.1 In-mem (NewInmemCluster)

Builds an N-node cluster over hashicorp/raft's InmemTransport and in-mem stores, fully interconnects the transports, bootstraps an identical N-server configuration into each node, and starts them. No network, etcd, or disk. This is the rung the in-process kill-the-leader integration test uses (HXC-1216): a real 3-node group elects a leader, and committed entries stay intact on the survivors after the leader is shut down.

2.2 Real TCP NetworkTransport (NewNetworkCluster, tcp.go)

`tcpStreamLayer` adapts a `*net.TCPLListener` into a hashicorp/raft `StreamLayer` (`Accept/Close/Addr/Dial`).

`openTCPTransportAt(id, bindAddr, base)` binds a real listener (resolving

bindAddr), wraps it in a `NewNetworkTransportWithConfig` (MaxPool: 3, Timeout: 5s), and returns the transport plus the concrete bound `*net.TCPAddr`. `openTCPTransport` is the ephemeral convenience over it ("127.0.0.1:0" → OS-assigned port). `newTCPNode` assembles a node from this transport with in-mem stores and TCP-tuned timers.

`NewNetworkCluster` keys the bootstrap configuration by each node's **real** bound TCP address, so peers reach each other by dialing genuine kernel sockets — `RequestVote/AppendEntries/InstallSnapshot` RPCs travel over real TCP. `NetTransport` and `LocalTCPAddr` accessors let tests assert each node bound a distinct non-zero port. `wrapNetworkNodes` owns leak-safe partial-failure cleanup: on any error it shuts down already-started nodes and closes the un-wrapped transports so no listener socket or goroutine leaks.

2.3 On-disk BoltDB persistence (`NewPersistentNode`, `store.go`)

`openPersistentStores(dataDir, base)` opens the durable storage half: a single raft-boltdb/v2 `BoltStore` (`raft.db`) that satisfies **both** `LogStore` and `StableStore`, plus a `FileSnapshotStore` under `snapshots/` (retain 2). It also **detects prior state** — a non-empty `raft.db` file or a `LastIndex() > 0` — and reports `alreadyInit` so a `reopen` does **not** re-bootstrap.

`NewPersistentNode(id, dataDir, base)` uses this with the in-mem transport (a single-node Raft is trivially leader; storage durability is orthogonal to the transport):

- **First call on a fresh dir:** bootstraps a single-server config into the durable stores; the node elects itself leader.
- **Later call on the same dir** (after the previous `Node.Shutdown` closed the bolt file): the node is **re-opened** — `hashicorp/raft` replays the persisted log / restores the latest snapshot into a fresh KVFSM, recovering all previously committed entries **without a caller re-applying them**.

This rung's `WaitForLeader(timeout)` additionally issues a Raft Barrier after leadership is observed, so on return every replayed-and-committed entry is already applied to the FSM and visible via `FSM().Get`.

2.4 Combined persistent + networked node (`persistnet.go`)

`persistnet.go` **composes** the two real primitives — without duplicating either — into one node that is both networked and durable. It reuses `openTCPTransportAt` (TCP) and `openPersistentStores` (BoltDB + snapshots), so a combined node carries a non-nil `netTransport` (TCP) **and** non-nil `dataDir/closeStore` (BoltDB):

- `NewPersistentNetworkNode(id, dataDir, base, servers)` — ephemeral-port convenience; returns a `PersistentNetworkNode` bundling the live `*Node`, its bound `*net.TCPAddr`, and (read-only via `LogStore()`) the on-disk store for direct log inspection.
- `NewPersistentNetworkCluster(base, ids, dataDirs)` — the persistent+networked analogue of `NewNetworkCluster`: binds each node's real TCP listener, bootstraps the full membership into every node's **durable** stores, and starts them. `wrapPersistentNetworkNodes` provides the same leak-safe partial-failure cleanup (closing both transports and bolt stores).

Reopen semantics match §2.3: on a fresh dir it bootstraps; on the same dir after a prior Shutdown (or an abrupt kill) it recovers the durable log/snapshot and rejoins **without** re-bootstrapping, binding a fresh listener.

2.5 Fixed-address multi-process primitive (NewPersistentNetworkNodeAt)

NewPersistentNetworkNodeAt(id, dataDir, bindAddr, base, servers) is the multi-**process** building block. Because the transport binds a **known fixed** address (e.g. 127.0.0.1:7001) instead of an ephemeral one, the full initial membership (servers — every node's id=addr, including self) can be configured ahead of time across separate OS processes. Each process calls it with the same servers list and its own id / bindAddr / dataDir:

- **First boot (fresh dir):** bootstraps servers into the durable stores.
- **Reopen (same dir, after a prior Shutdown or a SIGKILL):** recovers the persisted log/snapshot from disk and rejoins the live cluster **without** re-bootstrapping (the durable on-disk configuration wins; servers is ignored), using the same fixed bindAddr so peers reconnect by the address they know.

Node.Shutdown for such a node stops Raft, closes the TCP transport (releasing the listener), then closes BoltDB (flushing files) — in that order.

3. The helix-raftd daemon (cmd/helix-raftd)

helix-raftd is a single-process daemon hosting **one** persistent+networked node (real loopback TCP + real on-disk BoltDB) via NewPersistentNetworkNodeAt. Run three as separate OS processes — each with its own --data-dir and --bind, all sharing the same --peers membership — and they elect a leader, replicate over real TCP, and survive a leader-process kill + restart-from-disk. There is no in-process shim.

3.1 Flags (main.go)

Flag	Meaning
--id	This node's Raft server id; must appear in --peers.
--data-dir	Durable directory for this node's BoltDB (raft.db) + snapshots/.
--bind	This node's Raft TCP listen address (host:port), e.g. 127.0.0.1:7001.
--peers	Full initial membership including self : comma-separated id=addr pairs. Used only on first boot; on reopen the durable on-disk config wins.
--admin	This node's admin HTTP listen address (host:port), a separate port from --bind.

parseFlags validates that all four required flags are set, that --id appears in --peers, and that --bind **equals** the address recorded for that id in --peers (peers dial the membership address, so a node's advertised Raft address must be exactly what its peers will dial). parsePeers enforces non-empty, unique ids and addresses.

3.2 HTTP admin API (admin.go)

A small HTTP control surface on the --admin port, separate from the Raft TCP port, so a client can drive and observe the node out-of-band from Raft's own RPCs.

Endpoint	Method	Behaviour
/status	GET	JSON {id, state, isLeader, leader, leaderAddr, lastIndex, numKeys, applied} — live Raft state plus FSM sink-side counters. A supervisor polls this to detect “exactly one leader” and “follower caught up”.
/get?key=K	GET	Reads K from this node's FSM (no Raft round-trip). A GET served by a follower proves the value reached that follower's own state via replication.
/put?key=K&value=V	PUT/POST	Applies a SET through Raft . Leader-only: Raft replicates to a majority and applies to every FSM before the call returns. The value may be ? value= or the request body.

Leader-gated writes (no split-brain). On a follower, /put returns **421 Misdirected Request** with {error, leader, leaderAddr} so the client can redirect to the real leader. The same 421 path also covers a leadership change that races between the IsLeader() check and Apply (which returns ErrNotLeader). Only the leader ever mutates state, so there is no split-brain write.

3.3 Startup / graceful shutdown order (main.go)

Startup: bring up the persistent+networked node (binds TCP, opens BoltDB, bootstraps-or-recovers) → start the admin HTTP server (synchronous bind, so a bind failure surfaces before readiness is announced) → print a machine-parseable HELIX_RAFTD_READY id=... bind=... admin=... tcp=... line on stdout once both listeners are up.

On SIGINT/SIGTERM (10s timeout): **stop the admin server first** (no new requests), **then** Node.Shutdown (Raft → TCP transport → BoltDB, in that order, flushing the data dir so it can be reopened by a later process).

4. Durability and safety guarantees actually proven

4.1 Multi-process kill + restart-from-disk E2E (cmd/helix-raftd/e2e_test.go, HXC-982)

A real multi-**process** end-to-end test (build tag e2e; run with `go test -tags e2e -run TestE2E ./cmd/helix-raftd/ -v`). It is not a mock and not skipped: it `go` builds the daemon and spawns **3 genuine OS processes** over real TCP with real on-disk BoltDB, driving them via their admin HTTP ports. It proves, in order:

1. Exactly one leader is elected **across the processes** over real TCP.
2. A PUT on the leader is then served by a **GET on a follower process** (replication reached the follower's own FSM over TCP).
3. A PUT on a **follower** is rejected with 421 and the leader's address (leader-gated write).
4. The leader process is **SIGKILLed**; a **new leader** emerges among the two survivors and the value survives. A second key is PUT **while the old leader is down**.
5. The killed node is **restarted from its same data dir**; it rejoins, catches up `lastIndex`, and serves **both** keys (the persisted one and the one added while it was down).
6. **Decisive persistence proof**: the **full cluster** is stopped (every BoltDB flushes), each `raft.db` is asserted non-empty on disk, then **all three** are restarted from their same data dirs. A leader is re-elected and both values are readable on **every** process — they could only have come from on-disk BoltDB, not live replication.

The test writes timestamped evidence (`e2e.log` + a human-readable `README.md`) to `qa-results/helix-raftd/<run-id>/`.

4.2 TLC-verified snapshot / log-compaction safety (specs/RaftSnapshot.tla, HXC-979)

`specs/RaftSnapshot.tla` is a TLA+ model of the **snapshot-install / log-compaction safety** dimension the Go code exercises (`FileSnapshotStore` + log truncation past the snapshot point + `InstallSnapshot` to a follower whose log has fallen behind the leader's snapshot index, so a plain `AppendEntries` can no longer catch it up). It is deliberately not full Raft: it fixes one stable leader and models the `COMMIT` / `COMPACT` / `INSTALL` / `APPEND` actions of that dimension.

Two safety invariants are checked:

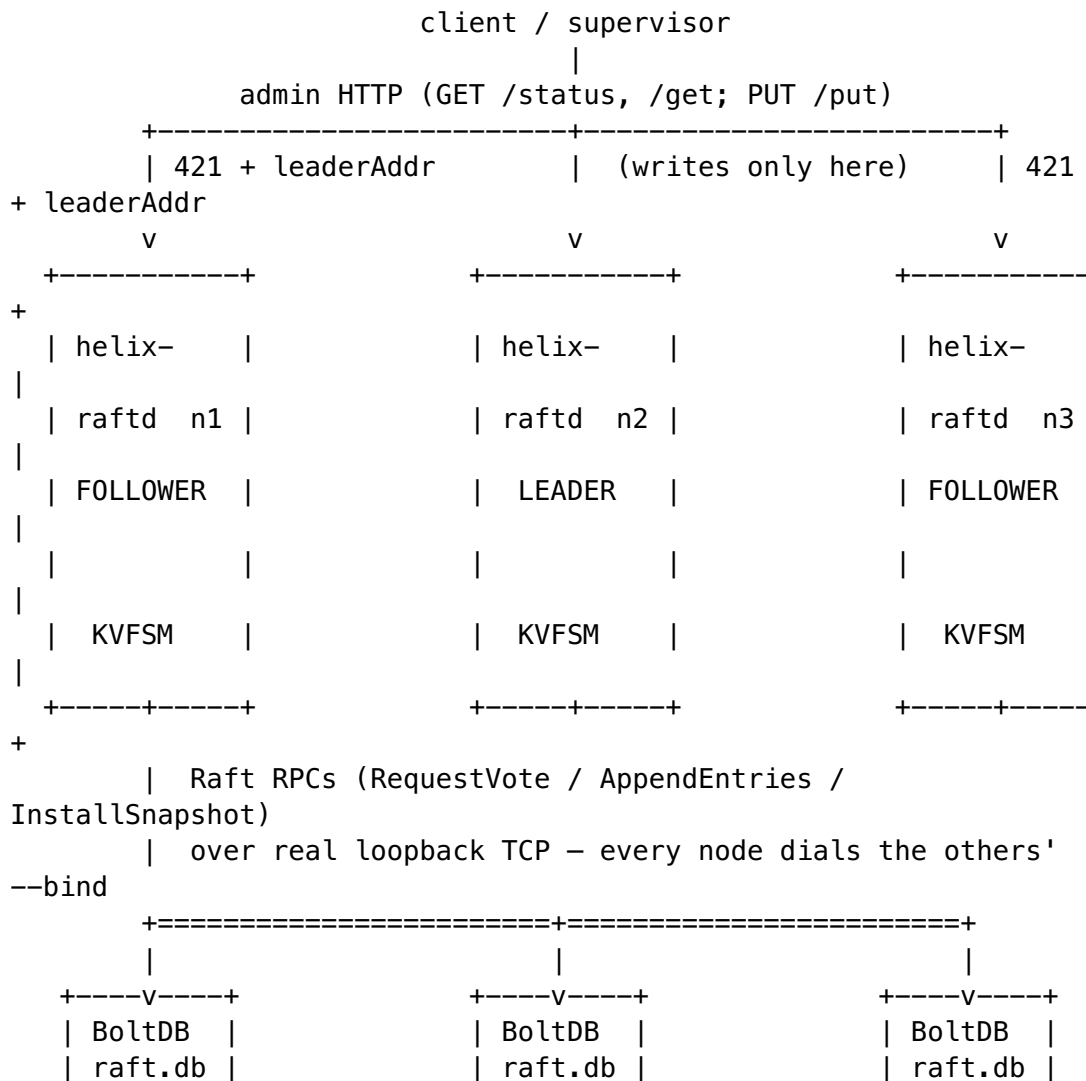
- **NoCommittedEntryLost** — every committed index is legitimately covered by the leader: either inside a snapshot whose point is itself $\leq$ `commitIndex` (reflects applied state) or still present in the un-truncated log.
- **SnapshotInstallCorrect** (`NoFollowerGap` $\wedge$ `FollowerAgrees`) — after a follower installs a snapshot and appends the rest, it holds every committed entry

up to its own frontier with no gap, and agrees with the leader on the shared committed prefix.

The model carries two mutation flags (`AllowBadCompact`, `AllowLossyInstall`) that, when enabled, deliberately break these invariants — the mutation-testing harness’s proof that the invariants have real teeth. The **main** configuration (`RaftSnapshot.cfg`) runs with **both mutations OFF**, `Followers = {f1, f2}`, `MaxIndex = 3` — the smallest domain that yields a genuine “compact past a follower’s log end, then install” gap. The bounded state space is explored **exhaustively to completion** (TLC reports 0 states left on the queue) under sound symmetry reduction over the interchangeable follower ids, with both invariants holding. (As checked locally with `tla2tools.jar`, the exhaustive run completes in a small, fixed number of states — on the order of a couple hundred — well within the time budget.)

5. A 3-node cluster (as deployed by helix-raftd)

Three `helix-raftd` processes, each binding a fixed Raft TCP port and a separate admin HTTP port, each backed by its own on-disk BoltDB. Raft RPCs flow over TCP between the nodes; clients drive the admin API on top; the leader is the only node that mutates state.



+snaps/	+snaps/	+snaps/
+-----+	+-----+	+-----+
data-dir n1	data-dir n2	data-dir n3

Which node is leader is decided by Raft election and changes on failure; the diagram shows one possible assignment. A write accepted by the leader is replicated to a majority and applied to every node's KVFSM before the leader's /put returns; each node's committed log and snapshots are durable in its own raft.db + snapshots/, so a node (or the whole cluster) recovers its state from disk on restart.

6. Source map

Concern	File(s)
Replicated FSM + Command	pkg/raft/fsm.go
Node contract, in-mem node builder	pkg/raft/node.go
Cluster helpers, in-mem cluster	pkg/raft/cluster.go
Real TCP transport	pkg/raft/tcp.go
On-disk BoltDB + snapshot stores, reopen-recovery	pkg/raft/store.go
Combined persistent+networked node, fixed-address builder	pkg/raft/persistnet.go
Daemon (flags, lifecycle)	cmd/helix-raftd/main.go
Daemon admin HTTP API	cmd/helix-raftd/admin.go
Prometheus /metrics endpoint + observability (docs)	cmd/helix-raftd/metrics.go
Multi-process kill+restart E2E	cmd/helix-raftd/e2e_test.go
Snapshot/compaction safety spec	specs/RaftSnapshot.tla, specs/RaftSnapshot.cfg